VULCANO

# Vulcano — documentazione tecnica

Come si avvia, si prova, si modifica e si pubblica il gioco. Per il funzionamento interno vedi ARCHITETTURA.md.

## Requisiti

- Un browser con **ES modules**, **Canvas 2D** e **AudioWorklet** (Chrome 66+, Firefox 76+, Safari 14.1+).
- Per sviluppo: **Python 3** (server statico) e **Node.js 18+** (controlli automatici, usa `structuredClone`). Nessuna dipendenza npm, nessuna build.

## Avvio in sviluppo

```
./start.sh                    # server + browser su http://localhost:8137/
./start.sh 9000 --no-open     # altra porta, senza aprire il browser
npm run serve                 # solo il server = python3 tools/serve.py 8137
```

`start.sh` controlla prima la porta: se risponde già Vulcano apre solo il browser, se risponde un'altra applicazione esce con errore invece di aprire la pagina sbagliata. Il server di `tools/serve.py` manda `Cache-Control: no-store`: senza, Chrome tiene in cache i moduli ES e un ricaricamento non vede le modifiche.

Serve un server HTTP: da `file://` i moduli e l'AudioWorklet non si caricano.

Il motore è nel submodule `lib/motorino` (repository locale `../motorino`): loop, input, font, sprite, mappe, fisica del giocatore, camera, particelle, stati, salvataggio, collegamento al SID. Si cambia lì, con i suoi test (`npm test`), poi qui `git -C lib/motorino fetch`, `checkout` della nuova versione, `git add lib/motorino` e un commit; dopo, `node tools/solve.mjs`.

Il SID è nel submodule `lib/sidicio` (MindContact/sidicio): clonare con `git clone --recurse-submodules`, o dare `git submodule update --init` in un clone già fatto. Senza, il gioco parte muto e `tools/check.mjs` fallisce sull'import.

Per cambiare l'emulazione o i player si lavora in sidicio (con i suoi test, `npm test` lì dentro), si fa commit e push nel suo repository, poi in Vulcano si registra la nuova versione con `git add lib/sidicio` e un commit. Prima e dopo si confrontano i WAV di `node tools/check.mjs --wav DIR`: un refactoring non deve cambiare un solo byte.

### Parametri URL

| Parametro | Effetto |
|---|---|
| `?level=N` | salta il titolo e parte dal livello N (1–5) |
| `?god` | immortale a lava, spuntoni, nemici e fuoco (non alle cadute); compare GOD nell'HUD |

Si combinano: `?level=3&god`.

### Dalla console del browser

Il gioco è esposto come `window.game`. Utile per i test:

```
game.state = 'play'                       // salta la scheda del livello
game.world.player.x = 60 * 16             // teletrasporto (pixel)
game.world.snapCamera()
game.audio.sfx('death')                   // suona un effetto
game.audio.replay('fuga')                 // suona un brano
localStorage.removeItem('vulcano.v1')     // azzera progressi e impostazioni
```

## Controlli automatici

```
npm test                            # mondo deterministico e replay dei livelli
node tools/check.mjs                # oppure: npm run check
node tools/check.mjs --wav DIR      # scrive anche ogni brano e gli effetti in WAV
node tools/solve.mjs                # tutti i livelli, a ogni difficoltà (oppure: npm run solve)
node tools/solve.mjs 3              # un livello
node tools/solve.mjs 3 hard         # un livello a una difficoltà (easy, normal, hard)
```

**`check.mjs`** verifica che:

- ogni sezione di ogni brano abbia le tre voci della stessa lunghezza e a battute intere (16 righe);
- ogni brano e ogni effetto si renda senza `NaN` e non sia muto;
- i jingle (brani senza loop) finiscano;
- ogni livello si analizzi (caratteri noti, un inizio, almeno un'uscita).

**`solve.mjs`** dimostra che ogni livello si può finire: una ricerca best-first su sequenze di input, eseguite sulla vera classe `World` (fisica, nemici, geyser, lava che sale, piattaforme), a ognuna delle tre difficoltà. Se un livello non si risolve stampa il punto più vicino all'uscita raggiunto; con `MAP=1` anche la mappa attorno. Va rilanciato **dopo ogni modifica a un livello, alla fisica del salto, alla tabella delle difficoltà o a motorino**. Ogni strada trovata finisce in `test/replays/`, e **`npm test`** la rigioca dall'inizio: se un replay non arriva più all'uscita, il risolutore va rilanciato. Lo stesso `npm test` controlla che il mondo sia deterministico (due corse uguali, stesso stato) e che un salvataggio della ricerca riporti indietro esattamente.

Il WAV di `--wav` è il modo più rapido per ascoltare un brano mentre lo si scrive, senza aprire il gioco.

## Struttura dei file

```
index.html              canvas, tasti touch, carica src/main.js
style.css               scala pixel-perfect, tasti touch
src/main.js             shell: stati, punteggio, vite, menu, opzioni, HUD
src/i18n.js             testi a schermo in italiano e inglese
src/icon.js             l'icona dell'app, disegnata come testo
src/game/levels.js      livelli come testo + parser
src/game/world.js       simulazione di un livello
src/game/difficulty.js  le tre difficoltà: vite, lava, nemici, golem
src/game/render.js      disegno del mondo
src/game/sprites.js     sprite pixel-art come testo
src/audio/songs.js      brani e strumenti in notazione tracker
src/audio/sfx.js        effetti sonori come programmi a frame
lib/motorino/           submodule: il motore 2D (loop, input, font, fisica, audio…)
lib/sidicio/            submodule: emulazione SID, player, compilatore, worklet
start.sh                avvio: server + browser
manifest.webmanifest    la PWA: nome, colori, icone
sw.js                   service worker (quello di motorino)
icons/                  icone dell'app, generate da src/icon.js
tools/serve.py          server di sviluppo senza cache
tools/check.mjs         controlli di audio e livelli
tools/sim.mjs           il mondo senza browser, per solver e test
tools/solve.mjs         prova di completabilità dei livelli
test/                   test (node:test) e replay dei livelli
docs/                   questa documentazione
```

## Modificare i contenuti

### Un livello

I livelli stanno in `src/game/levels.js`, disegnati come testo, un carattere per casella da 16×16 px. La legenda è in testa al file:

| Car. | Significato | Car. | Significato |
|---|---|---|---|
| `#` | roccia | `=` | crosta (solida solo dall'alto) |
| `C` | roccia che crolla | `~` | lava |
| `^` | spuntoni | `G` | bocchetta di geyser |
| `\|` | fermo invisibile per piattaforme | `o` | cristallo |
| `K` | checkpoint | `E` | uscita |
| `P` | partenza | `s` | lumaca |
| `v` | pipistrello | `f` | bolla di fuoco (in lava) |
| `M` | piattaforma orizzontale | `N` | ascensore verticale |
| `B` | golem di magma (boss) | | |

Il golem vuole un'arena larga 20 colonne che parte 9 colonne a sinistra di `B`; con il golem, le uscite del livello restano chiuse finché non cade. Conviene che l'arena sia l'ultima schermata della mappa (la telecamera, ai bordi, la mostra intera) e che ci sia un checkpoint appena prima.

I livelli orizzontali si compongono con `seg(altezza, righe)`, segmenti larghi 16 colonne allineati in basso, e `join(...)`; `roof(righe, n)` chiude di roccia le prime n righe (grotte). La voce in `LEVELS` dà nome, sfondo (`sky` o `cave`), brano, tempo di riferimento (`par`, secondi) e l'eventuale lava che sale (`rise: { delay, speed }`, in update e pixel per update) o il fronte di lava che insegue da sinistra (`chase: { delay, speed }`, stesse unità).

Misure da tenere a mente: il salto pieno sale di **circa 53 px (3⅓ caselle)** e copre circa **4 caselle** in orizzontale di corsa; tenere i buchi a 3 caselle al massimo. Poi: `node tools/check.mjs && node tools/solve.mjs && npm test`.

### Un brano

In `src/audio/songs.js` (formato di sidicio, vedi `lib/sidicio/README.it.md`), ogni brano ha `speed` (frame a 50 Hz per riga), `filter` opzionale (cutoff 0–2047, risonanza 0–15, modo 1=passa-basso, LFO sul cutoff), `sections` (ognuna con tre stringhe, una per voce) e `loop` (indice della sezione da cui ripetere, `-1` per i jingle).

| Token | Significato |
| --- | --- |
| `C4` `D#5` | nota (C4 = MIDI 60) con lo strumento corrente |
| `C4@lead` | una nota con un altro strumento |
| `C4~` | glide verso la nota senza riattaccarla |
| `.` | tieni |
| `-` | rilascia (gate off) |
| `K` `S` `H` | cassa, rullante, charleston |
| `@nome` | cambia strumento (non occupa una riga) |

Gli helper `bassEighths`, `bassGallop`, `bassSlow`, `drumChords`, `padChords`, `rest` generano una battuta per accordo. Gli strumenti stanno in `INSTRUMENTS`; i campi sono documentati in ARCHITETTURA.md.

### Un effetto sonoro

In `src/audio/sfx.js`: `prio` (un effetto ruba la voce solo a uno di priorità uguale o minore), `adsr`, `frames` (una voce `[onda, hz, larghezzaImpulso]` per frame a 50 Hz, `null` = gate off) e `cutoff` opzionale (un valore per frame, passa il suono nel filtro). Gli helper `sweep` e `notes` coprono quasi tutto. Poi chiamalo con `this.hooks.sfx('nome')` dal mondo o `this.audio.sfx('nome')` dalla shell.

### Uno sprite

In `src/game/sprites.js`, sprite come righe di testo: ogni carattere è un colore della `PALETTE`, `.` è trasparente. Tutte le righe della stessa larghezza (altrimenti errore al caricamento). Lo sprite guarda a destra; il renderer lo specchia.

Un personaggio nuovo: busto e gambe in `CHARACTERS`, nome nelle due lingue in `src/i18n.js` (`characters`). Tenere il riquadro 12×16: la hitbox è la stessa per tutti.

### Un testo a schermo

In `src/i18n.js`, la stessa chiave in entrambe le lingue, poi `this.t.chiave` nella shell. Solo maiuscole: il font non ha minuscole.

### La documentazione in HTML e PDF

Accanto a ogni `docs/*.md` ci sono il suo `.html` e il suo `.pdf`, generati:

```
./tools/docs.sh        # rigenera HTML e PDF di GUIDA, TECNICA, ARCHITETTURA
```

Usa il convertitore `md2pdf.sh` di md-to-pdf (sul `PATH`, oppure indicato con `MD2PDF=...`) e Chrome headless; lo stile è `tools/doc-template.html`. Rilanciarlo dopo ogni modifica a un documento, nello stesso commit. Il convertitore è volutamente piccolo: niente elenchi annidati né HTML nel Markdown.

## Salvataggio

Una sola chiave `localStorage`, `vulcano.v1`:

```
{ "settings": { "music": 7, "sfx": 8, "model": 6581, "lang": "it", "character": "explorer",
                "difficulty": "normal" },
  "progress": { "unlocked": 2, "best": 12400 } }
```

Il muto non si salva. Se il formato cambia in modo incompatibile, cambiare il nome della chiave.

## Pubblicazione

Il gioco è un sito statico: basta copiare `index.html`, `style.css`, `manifest.webmanifest`, `sw.js`, `icons/`, `src/`, `lib/motorino/src/`, `lib/motorino/pwa/` e `lib/sidicio/src/` su un qualunque hosting HTTPS (GitHub Pages, Netlify, un bucket). `tools/`, `docs/` e `package.json` non servono in produzione. L'AudioWorklet richiede un contesto sicuro: HTTPS o `localhost`.

### PWA

Vulcano si installa come app e gioca offline. `manifest.webmanifest` dà nome, colori, orientamento orizzontale e icone; `sw.js` importa il service worker di motorino (`lib/motorino/pwa/sw.js`), che chiede sempre i file alla rete e ne tiene una copia: offline serve l'ultima copia buona. Non c'è un elenco di file da aggiornare, perché il gioco carica tutti i suoi moduli all'avvio. In sviluppo non dà fastidio: con la rete vince sempre la rete.

Le icone in `icons/` (32, 180, 192 e 512 px) si rifanno dal disegno in `src/icon.js`:

```
node lib/motorino/tools/icon.mjs src/icon.js icons
```

## Prestazioni

Il costo vero è l'audio: due SID con oversampling 4x significano circa 350 000 passi oscillatore al secondo per chip, nel thread audio. Misurato in Node sulla macchina di sviluppo (tre voci per chip, filtro acceso): circa 120 ms di calcolo per secondo d'audio, cioè **~12% di un core**. È il primo posto da guardare se il gioco scatta su hardware debole; `OVERSAMPLE` in `lib/sidicio/src/chip.js` a 2 dimezza il costo a prezzo di un po' di aliasing negli acuti. Il disegno è a 320×180, scalato dal CSS con `image-rendering: pixelated`; la parte statica del livello è pre-renderizzata.